Introduction

Profile

정정교 Jung Junggyo

1998.10.20

📧 이메일 : willy_1020@naver.com

🔗 Github : github.com/junggyo1020

📝 블로그 : willie의 작은 공간

안녕하세요. 주니어 백엔드 개발자 정정교입니다.
단순한 기능 구현을 넘어서, **실제 사용자가 직면한 문제를 깊이 이해하고 그에 맞는 기술적 솔루션을 설계하는 것을 지향**합니다.

3000명 규모의 동아리 모임 서비스, 1700명 실사용자를 보유한 대학생 인턴 플랫폼,
소비자와 소상공인을 연결하는 쿠폰관리 서비스 등
다양한 도메인에서의 개발 경험을 통해 각 비즈니스 영역의 특성을 파악하고,
해당 도메인에 최적화된 구조와 솔루션을 제공해왔습니다.

새로운 기술을 학습하는 것에 두려워하지 않습니다.
꾸준히 의미있는 활동을 통해 성과를 만들어가는 개발자가 되겠습니다.

Tech Stack

Java ■■■■□	객체지향 설계(OOP) 기반 서버 로직 구현, Stream API 및 람다 활용, 10개 이상의 프로젝트를 통한 비즈니스 로직 최적화 경험
Spring Boot ■■■■□	REST API 설계 및 도메인 주도 설계(DDD) 적용, 파사드 패턴 기반 계층 분리, JWT 인증/인가 및 트랜잭션 관리 체계 구축
RDBMS ■■■■□ (MySQL / PostgreSQL)	서비스 요구사항에 최적화된 데이터 모델링 및 스키마 설계, 실행 계획 분석을 통한 인덱스 최적화 및 쿼리 성능 개선
JPA / QueryDSL ■■■□□	캐싱 전략을 통한 DB 부하 감소 및 조회 성능 개선, 분산 락(Distributed Lock) 구현을 통한 데이터 정합성 및 안정적인 트래픽 처리
Infrastructure (Linux / CI/CD) ■■■□□	Docker 및 Jenkins/GitHub Actions 기반 CI/CD 파이프라인 구축, Nginx를 활용한 무중단 배포(Blue-Green) 환경 운영 경험

Introduction

Education

SSAFY (삼성청년SW·AI아카데미) 14기 25.07.08 ~ 26.06.30

한양대학교 컴퓨터소프트웨어학부 졸업 25.08.21

고려대학교 경제통계학부 빅데이터전공 중퇴 23.01.31

휘문고등학교 졸업 17.02.28

Awards

SSAFY 자율 프로젝트 수상(우수상) 26.05.21

SSAFY 공통 프로젝트 수상(우수상) 26.02.19

NOW SOPT 34th APPJAM Demoday 수상(최우수상) 24.07.27

FORIF 23 Winter Hackathon 1위 24.02.08

고려대학교 학장상 22.08.25

고려대학교 성적 최우수 학생 표창장 19.07.09

707 대테러 종합평가 최우수 요원 선정 (707 특수임무단장상) 21.06.11

고려아카데미컨설팅 프로젝트 대상 19.06.05

Certifications

TOEIC SPEAKING IH (150) 26.02.28

정보처리기사 25.09.12

SQLD 25.04.04

ADSP 26.06.05

투자자산운용사 26.01.29

컴퓨터활용능력 1급 21.12.17

빅데이터분석실무 2급 19.06.17

ICDL COMPUTING(Python) 19.07.05

Experiences

하나금융티아이 금융 IT 서비스 개발 및 운영 2026.09 ~ 현재

삼성 사장단 및 임원단 대상 AX교육 보조강사 2026.06

SOPT Makers 36기 Playground Team 2025.02 ~ 2025.08

SSAFY CS 스터디 그룹장 2025.07 ~ 2025.11

SSAFY 알고리즘 스터디장 2025.07 ~ 2025.11

SOPT 디자인 패턴 스터디 2024.05 ~ 2024.06

SOPT 34기 Server Part 2024.03 ~ 2024.07

UMC 5기 Server Part 2023.09 ~ 2024.02

한양대학교 중앙 프로그래밍 동아리 ForIF 2023.09 ~ 2023.12

한양대학교 알고리즘 동아리 ALOHA 2023.03 ~ 2024.02

교내 비교과 서포터즈 잡아라서포터즈 4기 2019.09 ~ 2020.01

GDS 컨설팅 그룹 인턴 2019.12 ~ 2019.12

빅데이터·국가통계 전공 학생회 사무국장 2019.03 ~ 2019.12

공공정책대학 학생회 집행부장 2019.03 ~ 2019.12

전공 스터디 SCR 회장 2019.03 ~ 2019.12

밴드 소모임 청울림 회장 2019.03 ~ 2019.12

파이썬 스터디 BigPy 팀장 2019.03 ~ 2019.07

중앙 마케팅 동아리 MMC 2019.03 ~ 2019.06

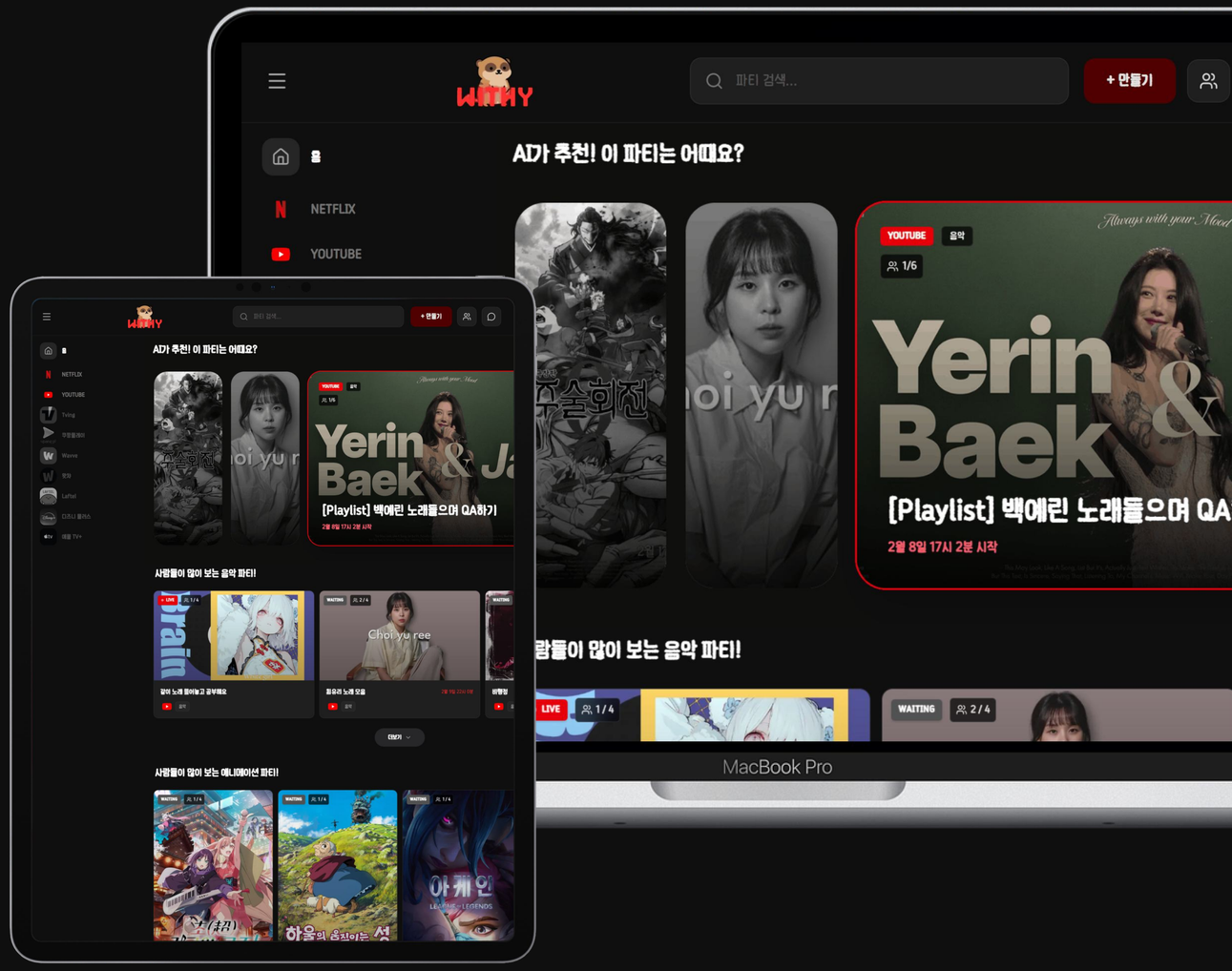
장애학습지원센터 학습도우미 2018.10 ~ 2019.10

교내 학습지원 비상서포터즈 6기 2018.09 ~ 2018.12

WITHY

AI 클린봇 '실시간 영상 같이 보기' 서비스, 위디 Withy

욕설, 스포일러를 감지하는 AI 클린봇으로 쾌적한 영상 같이 보기 서비스를 개발하여, **프로젝트 150팀 중 최우수팀**으로 선정되었습니다



소속 : SSAFY (삼성청년 SW · AI 아카데미) 부트캠프 14기 공통 프로젝트

수상 : 본선 발표회 최우수팀 선정, '우수상' 수상

구성 : 팀 프로젝트 (BE 2명, FE 3명, AI 1명)

기여도 : 40% 이상

상세 역할 : 위디(Withy)의 팀 리더 & 백엔드 개발 & 서비스 기획 & AI를 활용한 디자인

Github : https://github.com/Watch-with-WITHY/WITHY-SERVER

진행 기간 | 2026.01.06 ~ 2026.02.09 (5주)

Introduction

프로젝트 소개

욕설 및 스포일러 AI 감지와 실시간 영상 동기화 기술을 결합한 같이 보기 서비스

기획 배경

저작권 문제로 넷플릭스, 유튜브 **화면 공유가 불가능**했고,

같이 보는 채팅은 **욕설**과 **스포일러**로 시청 경험을 자주 해쳤습니다.

두 문제를 동시에 해결하는 **클린한 같이 보기 서비스**를 만들고자 기획했습니다.

주요 기능 *담당 기능

- ***AI 클린 채팅** - 욕설 실시간 필터링, 스포일러 감지 및 차단
- ***실시간 영상 동기화** - 화면 공유 없이 OTT 타임라인 정밀 동기화
- ***AI 맞춤 추천** - 사용자 취향, 시청 이력 기반 개인화 파티 추천
- 1:1 다이렉트 메시지 - 퇴장 시점 기반 대화 히스토리 필터링
- 통합 검색 - 파티, 콘텐츠, 호스트 통합 검색

주요 기여

- PM & 백엔드 리더로서 기술 방향 결정 및 팀 일정 전체 관리
- gRPC 도입 및 Kafka 비동기 파이프라인 설계로 Throughput **25%** 향상, 평균 지연 **32%** 개선 (517ms → 356ms)
- 서버 시간 보정 공식 설계로 영상 동기화 오차 **12ms 이내** 달성
- Context Caching 도입으로 AI 추천 응답 비용 **90%** 절감

기술 스택

Backend	Java 17, Spring Boot 3, Spring Security, JPA, QueryDSL
Communication	gRPC (Protobuf), WebSocket (STOMP)
Database	MySQL 8, Redis
Messaging	Apache Kafka
Auth	Kakao OAuth2, JWT
Storage	AWS S3
Infrastructure	Docker, Docker Compose, Jenkins, Nginx

주요 기술

gRPC : REST의 텍스트 파싱 오버헤드 문제 해결

→ **HTTP/2 + Protobuf 바이너리 통신**으로 latency 개선, AI 서버와 실시간 비속어 판별 통신

Kafka : 스포일러 감지 연산이 메인 서버에 영향 주는 구조 개선 필요

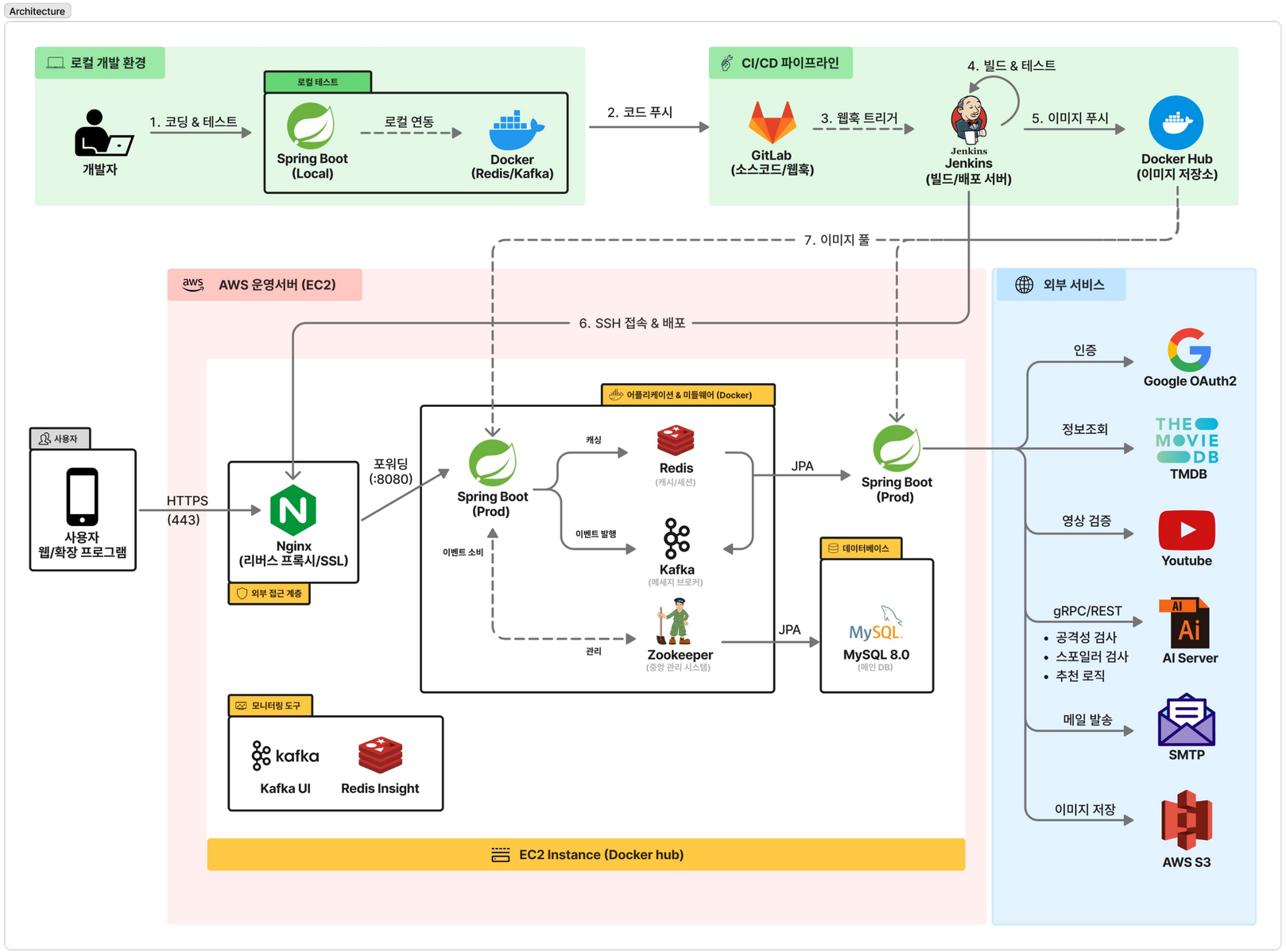
→ AI 분석 모듈 비동기 분리, Back pressure 제어

Redis Lua Script : 메시지 추출 연산의 원자성 보장 필요

→ 채팅 버퍼 원자적 추출

Context Caching : 추천 요청마다 전체 줄거리 전송하는 비효율 개선 필요

→ 파티 생성 시점에 1회 캐싱 인풋 토큰 90% 절감, TTFT **O(N)→O(1)**



인프라 아키텍처 설계

확장성과 가용성을 고려하여 이벤트 기반 비동기 아키텍처를 설계했어요

이벤트 기반 비동기 처리 및 서비스 디커플링 (Kafka & Redis)

- 핵심 도메인 로직과 부하가 큰 AI 분석 모듈을 Kafka로 분리하여 시스템 결합도를 낮추고, Redis를 활용한 캐싱/세션 관리로 데이터 액세스 성능을 극대화했습니다.

보안 및 고성능 외부 서비스 통합 게이트웨이 (Nginx & gRPC)

- Nginx 리버스 프록시를 통한 SSL 종단 및 보안 계층을 구축하고, 외부 AI 서버와 gRPC 기반의 고성능 바이너리 통신을 수행하여 실시간 처리 지연을 최소화했습니다.

CI/CD 자동화 파이프라인 및 모니터링 체계 구축

- Jenkins/Docker 기반의 Full CI/CD 파이프라인을 구축하여 배포 생산성을 높였으며, Kafka UI와 Redis Insight를 도입하여 분산 환경에서의 실시간 모니터링이 가능하도록 하였습니다.

문제 해결 01 : REST의 한계를 gRPC 도입을 통해 개선하여 실시간성 확보

WITHY

초당 100회 이상의 채팅을 보내도 '최대 지연 시간'을 1.4s → 0.5s 로 개선했어요

*최악의 응답(Tail Latency)이 0.5s

Problem 문제

기존 REST API(JSON)의 텍스트 파싱 오버헤드로
실시간 비속어 검사 시 Latency 발생.

Solution 해결

HTTP/2 + Protobuf 기반의 gRPC 도입.

Metrics 성능 개선

- K6 부하 테스트 결과 Throughput 25% 향상
- 평균 지연 시간 32% 개선 (517ms → 356ms)
- 최대 지연 시간 63% 개선 (1.4s → 0.5s)

REST API vs gRPC

REST API (Average) 517ms
JSON OVERHEAD

gRPC (Average) 350ms
OPTIMIZED

평균 지연 시간 개선율
32%
517ms → 350ms 개선

REST API (Max) 1,400ms
WORST CASE OVERHEAD

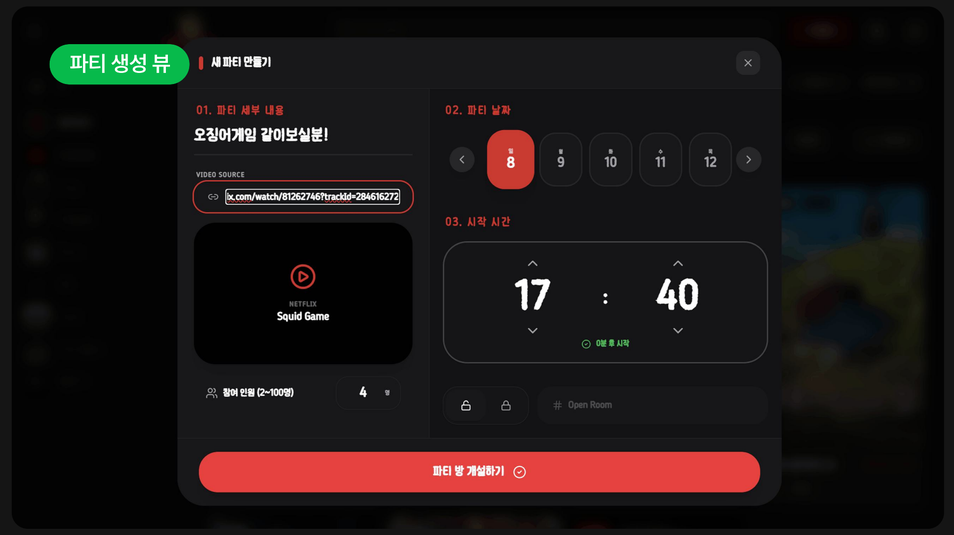
gRPC (Max) 554ms
STABILIZED

최대 지연 시간 개선율
63%
1,400ms → 554ms 개선

문제 해결 02 : Context Caching 도입으로 AI 운영 효율 및 비용 절감

인풋 프롬프트 토큰 비용 90% 절감,
O(N) → O(1) 수준으로
추천 응답 속도(TTFT)를 개선했어요

WITHY



Problem 문제

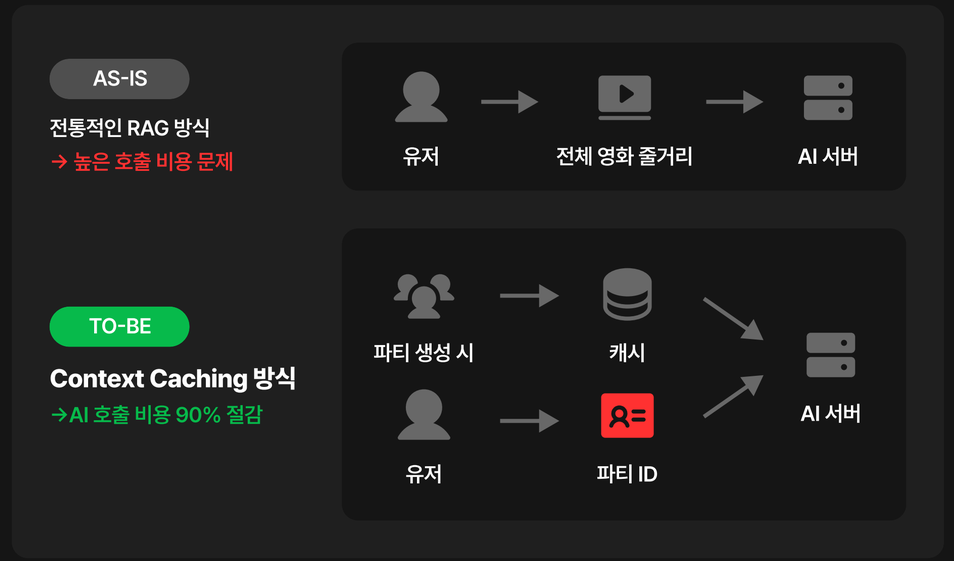
추천 요청 시마다 반복되는 방대한 영화 줄거리 데이터 전송(토큰 비용 발생).

Solution 해결

파티 생성 시점에 영화의 줄거리를 미리 캐싱하는 Context Caching 도입.

Metrics 성능 개선

- 인풋 프롬프트 토큰 90% 절감
- O(N) → O(1) 수준으로 추천 응답 속도(TTFT) 개선.

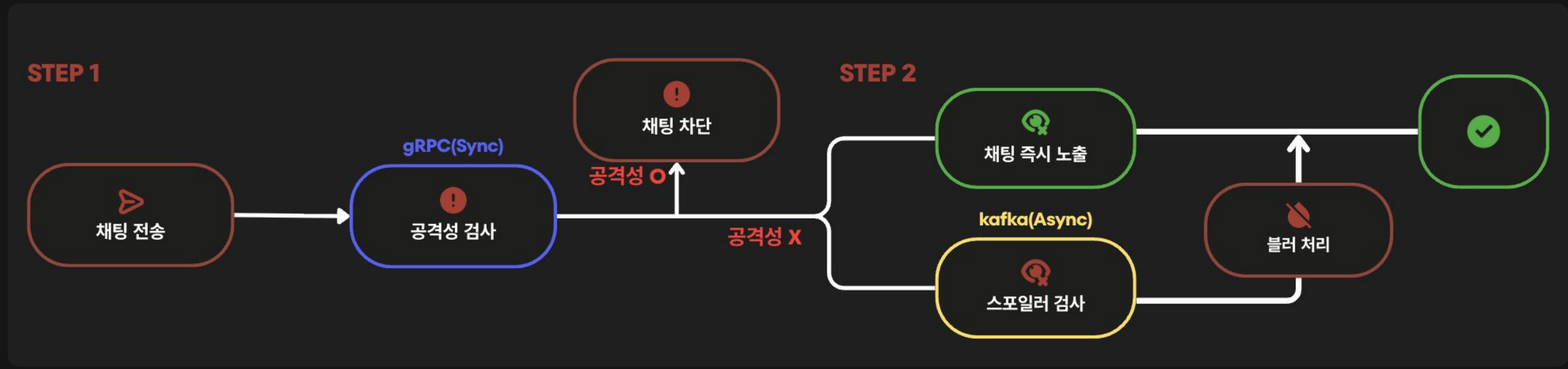


문제 해결 03 : Kafka와 Redis를 활용한 서비스 가용성 개선

WITHY

AI 서버 장애 발생 시에도 채팅 유실 0건, 메인 서비스 가용성 100% 유지했어요

Problem 문제	Solution 해결	Technical Detail 사용한 기술
연산 부하가 큰 스포일러 검사 로직이 메인 서버에 영향을 주는 구조.	Kafka를 통한 서비스 디커플링 및 Back pressure 제어	Redis Lua Script를 활용한 메시지 추출 로직의 원자성 보장.



문제 해결 04 : SSOT 기반 설계를 통한 서버 사이드 초저지연 동기화

WITHY

50명 동시 시청 환경에서 호스트가 시청 구간을 바꿔도 시청자는 시청 시점이 12ms 안에 동기화되도록 개선했어요

호스트가 바꾼 시청 구간

시청자가 보는 실시간 화면 → 실시간으로 같이 보기 완벽 구현

Problem 문제

클라이언트의 다른 로컬 시간(Clock Drift)으로 인해
싱크가 어긋나는 문제

Solution 해결

서버 시간 기준의 보정 공식 설계

Technical Detail 사용한 기술

Actual Position =
Saved Position
+ (Current Server Time - Last Updated Time)

문제 해결 05 : 팀 리더이자 PM으로서의 리스크 관리 성공

WITHY

기술 방향, 이슈 관리, 일정, 발표까지

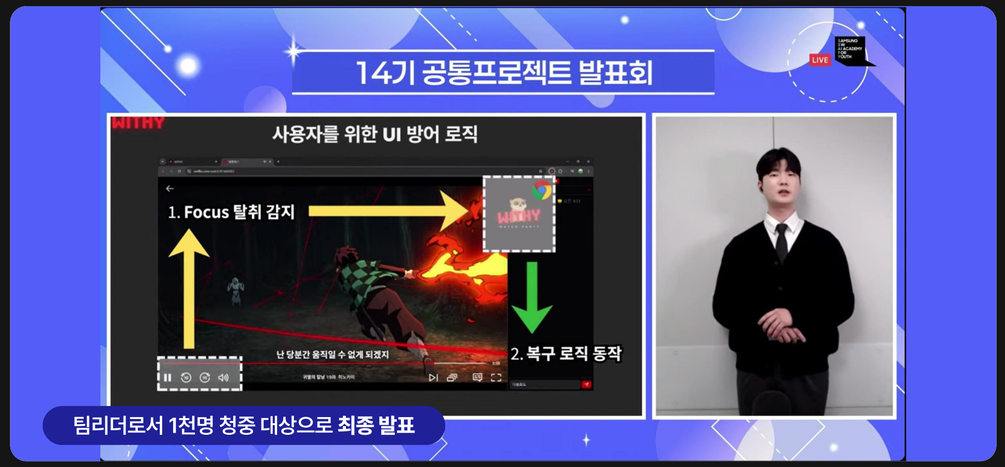
팀 리더이자 PM으로서 팀원들과 함께 프로젝트를 완성했어요

팀원 정보

역할	이름	이슈 현황				리뷰 현황		
			진행	완료		완료	취소	참여일
팀장	정정교	0	0	140	0	13	0	-
팀원	강영욱	0	0	44	0	15	0	-
팀원	김건희	0	0	36	1	3	0	-
팀원	송영주	0	0	35	1	2	0	-
팀원	이민엽	0	0	47	0	16	0	-
개발 리더, 기여도 40% 이상		0	0	35	0	5	0	-

정정교

140



이슈현황

100 % 진행률

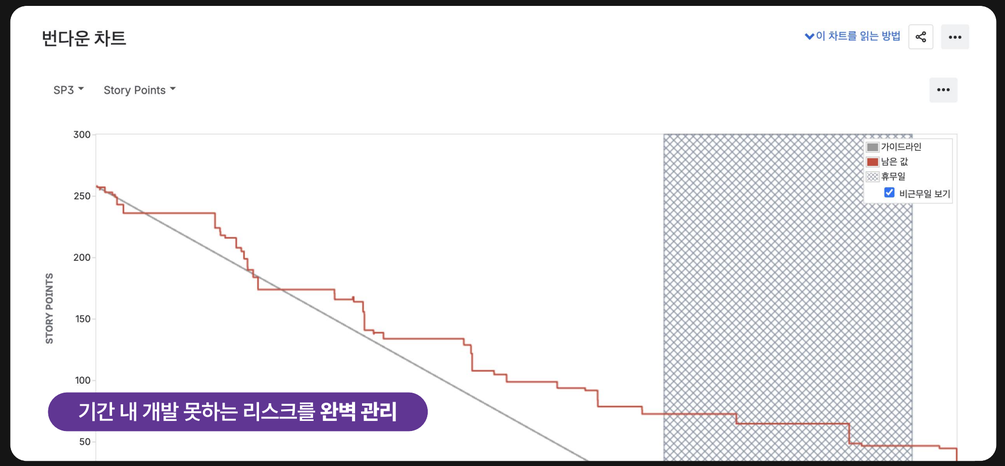
할일 0 진행 0 완료 339 전체 339

정렬기준 중요도 유형 전체

나의요청 상태 전체 할일

키워드 검색어를 입력해 주세요.

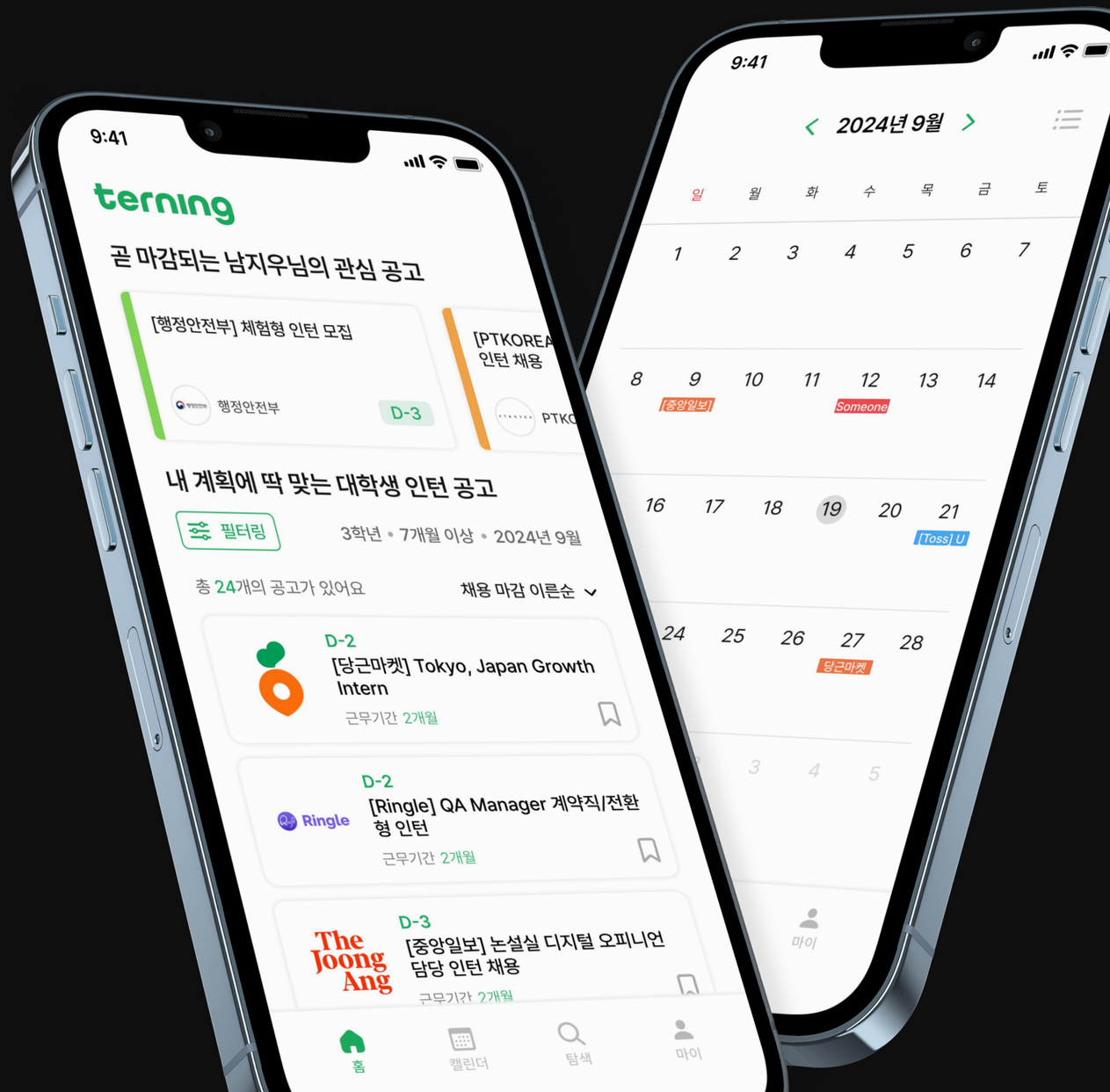
No	상태	유형	중요도	Epic	제목	담당자	등록자	
1	완료	Story	상	-	채팅 UI 수정	송영주	강영욱	2026.
2	완료	Story	상	-	홈 화면 UI 수정	이민엽	강영욱	2026.
3	완료	Story	상	-	전체 UX 수정	이민엽	강영욱	2026.
339개 모든 이슈 해결 & 성능 개선					로그인/온보딩 UI 수정	강영욱	강영욱	2026.
5	완료	Epic	상	-	[공통] 최종 발표 준비	정정교	정정교	



진행 기간 | 2024.06.17 ~ 2025.10.31 (약 1년 5개월)

인턴 경험이 필요한 대학생을 위한 채용 플랫폼, 터닝 Terning

API 응답 속도 61% 단축,
요청 처리량 2.5배 향상시킨
앱 서비스를 1년 5개월 간 운영하며
1,700명의 유저를 얻었습니다



소속 : 전국 최대 규모 대학생 IT 창업 동아리 'SOPT' 34기

수상 : 8주 해커톤에서 '최우수상' 수상

협업 : 기획자, 디자이너, 개발자들의 팀 프로젝트 (기획 2명, 디자인 2명, 서버 3명, 앱 iOS 3명, 앱 And 4명)

기여도 : 인프라 100%, 백엔드 개발 50% 이상

상세 역할 : 백엔드 리더 & 인프라 담당

Github : https://github.com/teamterning/Terning-Server

Behance : Terning Behance 바로가기

Introduction

프로젝트 소개

대학생 인턴 공고 탐색부터 지원 일정 관리까지 제공하는 대학생 맞춤형 인턴 플랫폼

기획 배경

대부분의 채용 플랫폼은 **졸업자, 경력자 위주**라 대학생이 인턴 공고만 따로 모아보기가 어려웠고,
공고를 찾더라도 지원 마감과 면접 일정을 한눈에 **관리할 수단이 없어** 불편함을 겪었습니다.
두 문제를 동시에 해결하는 **대학생 인턴 전용 서비스**를 만들고자 기획했습니다.

주요 기능 *담당 기능

- ***인턴 공고 탐색** - 직무, 기간 등 맞춤 필터링으로 인턴 공고만 탐색
- ***캘린더 일정 관리** - 스크랩 공고를 색상별 캘린더로 한눈에 관리
- ***푸시 알림** - 마감 임박 공고 및 새 공고 자동 알림
- 인기 공고 큐레이션 - 조회수, 저장수 기반 트렌드 공고 제공

주요 기여

- 백엔드 리더로 API 설계, CI/CD 및 무중단 배포 인프라 전담 (기여도 **50%** 이상)
- QueryDSL Tuple 기반 Left Join으로 N+1 해결, API 응답 **61%** 단축, 처리량 **2.5배** 향상
- 알림 서버를 AWS Lambda 기반 별도서버로 분리해 메인 서버 장애 격리 구조 구현
- PostgreSQL 스키마 단위 분리로 스테이징 DB 비용 **70%,** 전체 운영비 **40%** 절감

기술 스택

Backend	Java 21, Spring Boot 3 Spring Security, JPA, QueryDSL
Database	PostgreSQL 42.7.3
Notification	FCM (Firebase Admin 9.2.0), AWS Lambda
Auth	Kakao OAuth2, JWT
Infrastructure	Docker, GitHub Actions, Nginx (Blue/Green), AWS EC2
Monitoring	Discord Webhook

주요 기술

QueryDSL : 공고 조회 시 스크랩 여부 확인으로 N+1 쿼리 발생
→ **Tuple 기반 Left Join 단일 쿼리**로 해결, 응답 61% 단축, 처리량 2.5배 향상

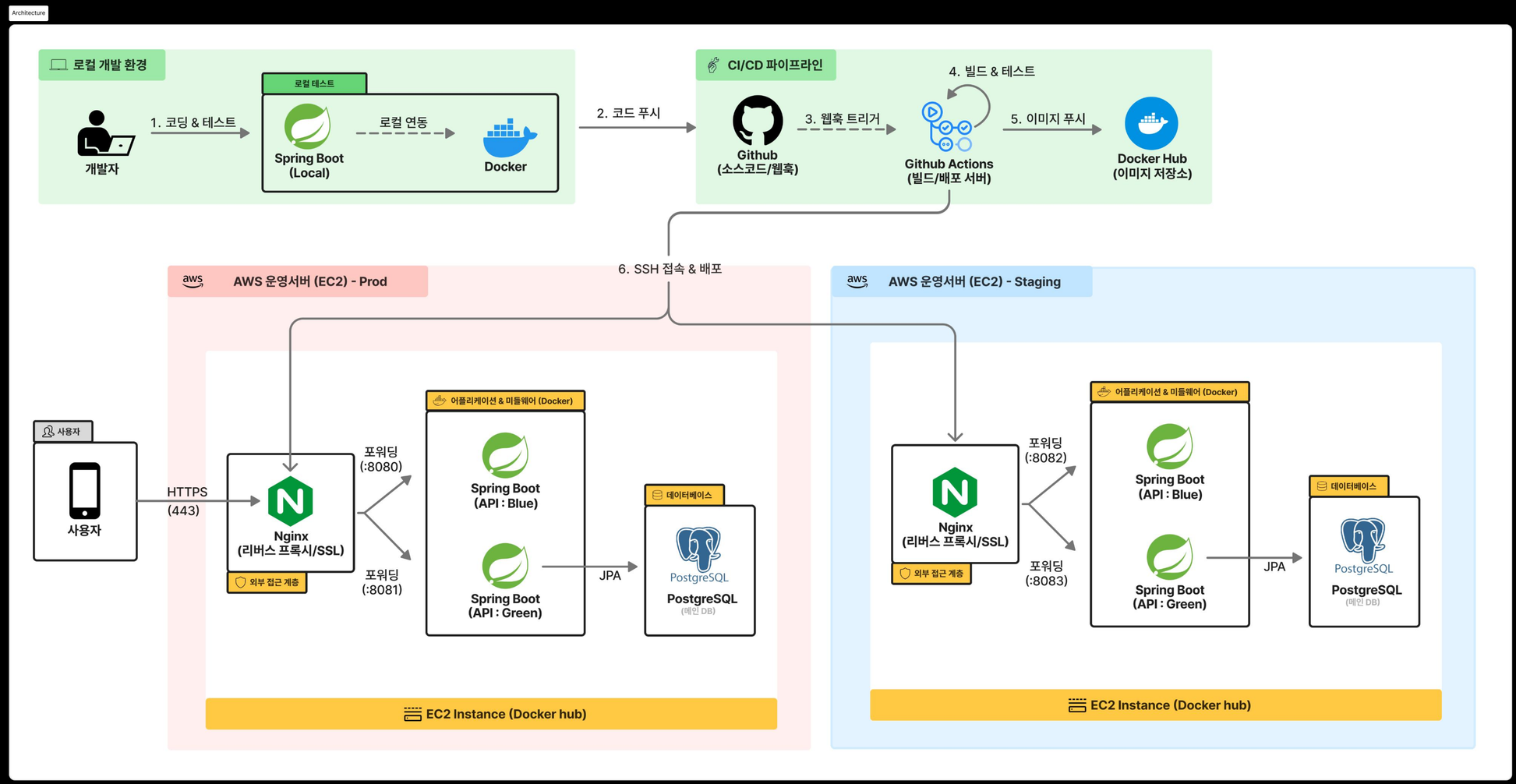
AWS Lambda : 상시 실행 불필요한 알림 스케줄러를 메인 서버에서 운영하는 비효율
→ **서버리스 Lambda로 분리**해 비용 절감 및 알림 서버 장애 격리

PostgreSQL 스키마 분리 : 스테이징 환경을 위한 별도 RDS 신설 시 비용 부담 증가
→ **단일 인스턴스 내 스키마 논리 분리**로 DB 비용 70%, 전체 운영비 40% 절감

Nginx + Blue/Green : 신규 버전 릴리즈 시 서비스 다운타임 발생 위험
→ **무중단 블루/그린 배포 적용**으로 1년 5개월 운영 중 오류로 인한 다운타임 Zero

인프라 아키텍처 설계

terning



무중단 배포 및 리버스 프록시 (Nginx & Blue/Green)

Nginx를 활용하여 SSL 종단 및 리버스 프록시 보안 계층을 구성하고, 무중단 블루/그린(Blue/Green) 배포 전략을 적용하여 신규 버전 릴리즈 시 발생하는 다운타임을 원천 차단했습니다.

컨테이너 기반 CI/CD 자동화 파이프라인 구축 (GitHub Actions & Docker)

GitHub Actions 기반의 CI/CD 파이프라인을 설계해 소스코드 빌드, 테스트, Docker Hub 이미지 푸시, EC2(Prod/Staging) 서버 배포까지의 모든 과정을 자동화하여 개발 생산성을 높였습니다.

비용 최적화 및 안전한 테스트를 위한 스키마 분할(PostgreSQL)

스테이징 환경을 위한 별도의 분리된 물리 데이터베이스(RDS)를 신설하는 대신, 단일 PostgreSQL 인스턴스 내에서 스키마 단위 논리적 파티셔닝(prod/staging)을 구현해 데이터 충돌 없이 인프라 운영 비용을 70% 절감했습니다.

문제 해결 01 : QueryDSL을 활용한 Tuple 기반 성능 튜닝

terning

N+1 쿼리 최적화로 API 응답 속도 61% 개선, 처리량 2.5배 향상

Problem 문제

홈 공고 조회 시, 스크랩 여부 확인을 위해 공고 수(N)만큼 추가 쿼리 발생.
(N+1 문제 발생)

Reasons 의사결정 과정

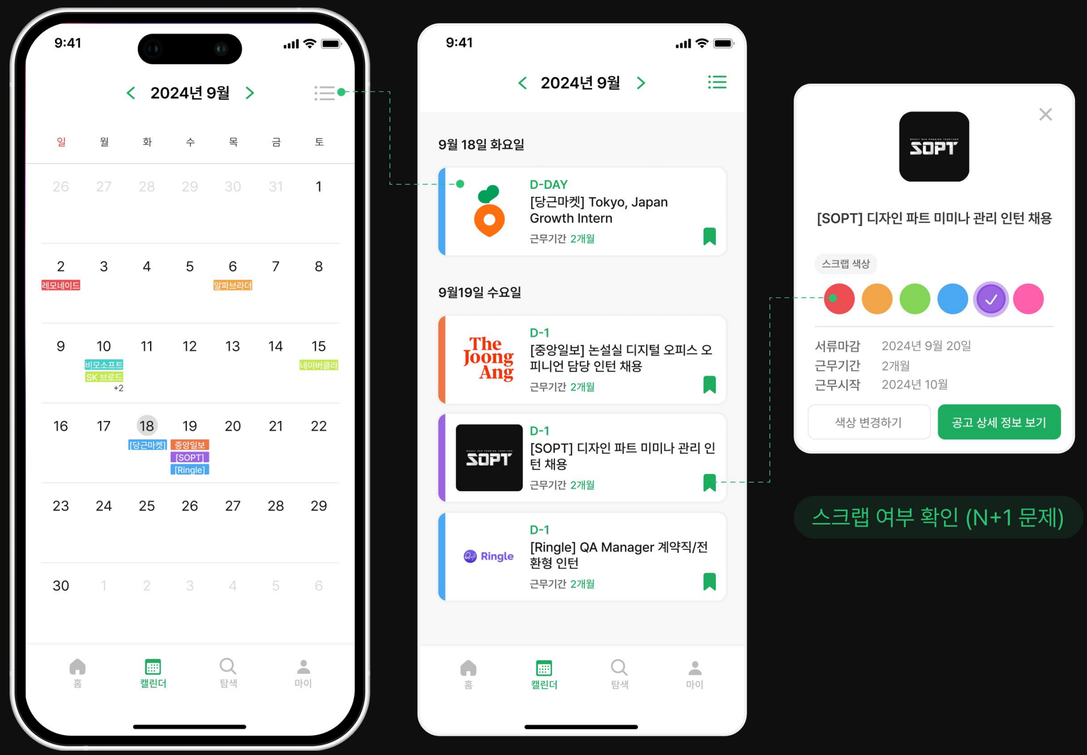
(고민) JPA 기본 메서드 vs QueryDSL
(결정) N+1 문제 해결 및 동적 쿼리 관리를 위해 QueryDSL 도입 결정.

Solution 해결

QueryDSL + Tuple 사용으로 공고 + 스크랩 정보를
단 한번의 Left Join 쿼리로 조회

Metrics 성능 개선

JMeter 기준 평균 응답 시간 61% 감소, 처리량 2.5배 향상



문제 해결 02 : 푸시 알림 시스템 분리 및 스케줄링 설계

terning

알림 서버(MSA) 분리로 장애 격리 및 안정성을 확보했습니다

Problem 문제

기존 아키텍처에서 알림 로직이 운영 서버와 강하게 결합 시,
핵심 기능(공고 조회 등)까지 알림 장애가 전파될 위험 존재.

Reasons 의사결정 과정

(고민) Spring @Scheduled vs AWS Lambda
(결정) 상시 필요 없는 스케줄러는 서버리스(AWS Lambda)가 비용 및 관리 측면에서 유리

Solution 해결

'알림 서버'를 별도 애플리케이션으로 완전 분리.
Lambda가 알림 서버를 호출하는 구조로 설계하여 장애 격리 구조 설계.

Metrics 성능 개선

서비스 전체의 안정성 및 향후 알림 채널(SMS 등) 확장성 확보.

문제 해결 03 : 스키마 단위로 스테이징 DB를 분리하여 유지비 절감

DB 비용 70% 및 전체 운영비 40% 절감

Problem 문제

스테이징 서버 구축과정에서 별도의 DB(RDS) 분리 시 비용 부담 증가.

Solution 해결

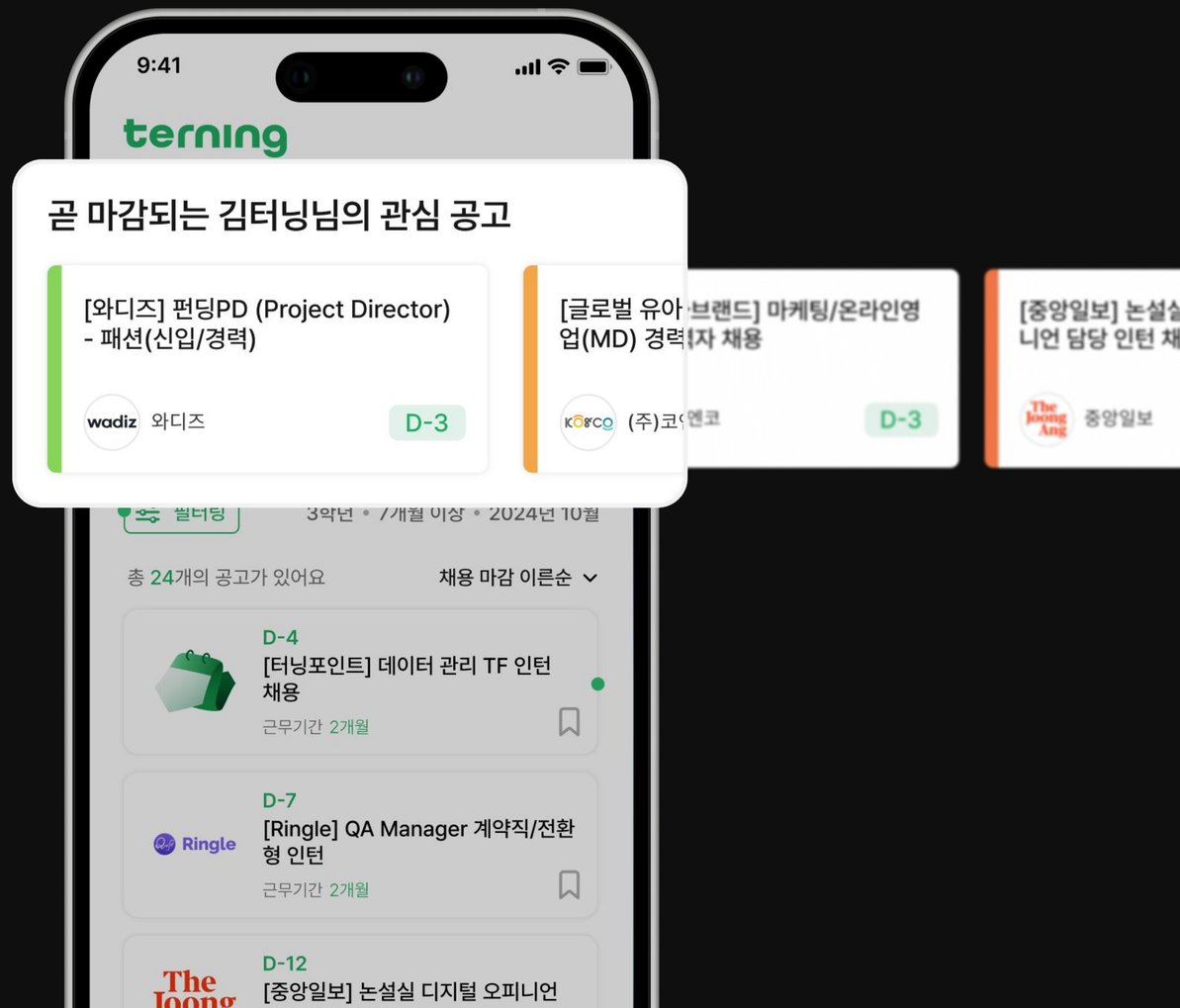
인스턴스 분리 대신 스키마 단위 분리 전략 선택 (Spring Profile 활용)

Result 성과

DB 비용 70% 및 전체 운영비 40% 절감.

Reference 참고

블로그 링크 🔗 스테이징 서버 구축하기



릴리즈 후 서비스 운영

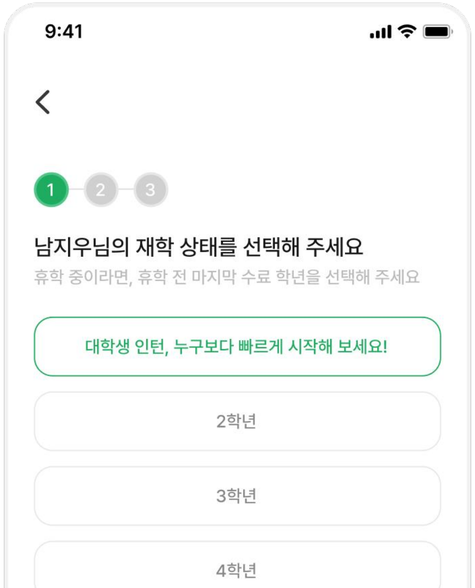
terning

유저테스트 UT를 4번 거듭하여,
1년 5개월 동안 사용자에게 꼭 필요한 기능을 추가했습니다

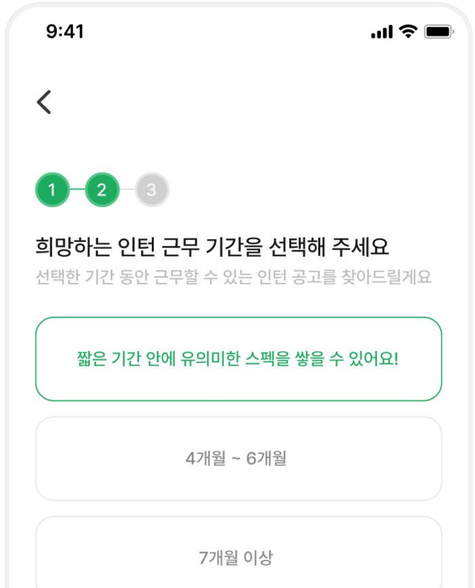
필터 건너뛰기 또는 중복 선택 기능이 필요해요
미래 공고는 데이트 피커에서 삭제해주세요

온보딩 개선
: 건너뛰기 기능 추가, 데이트 피커 범위 수정

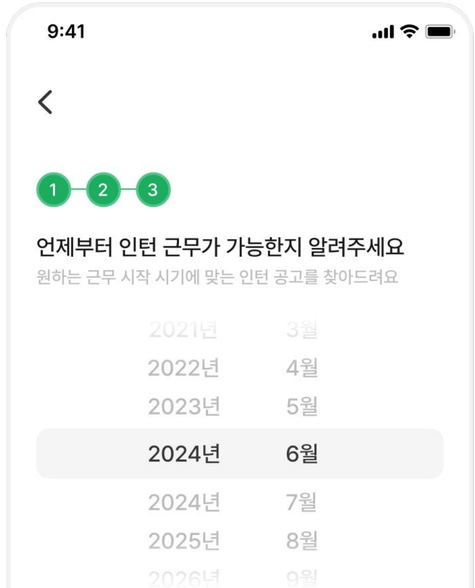
재학 상태



희망 근무 기간



근무 가능 시기



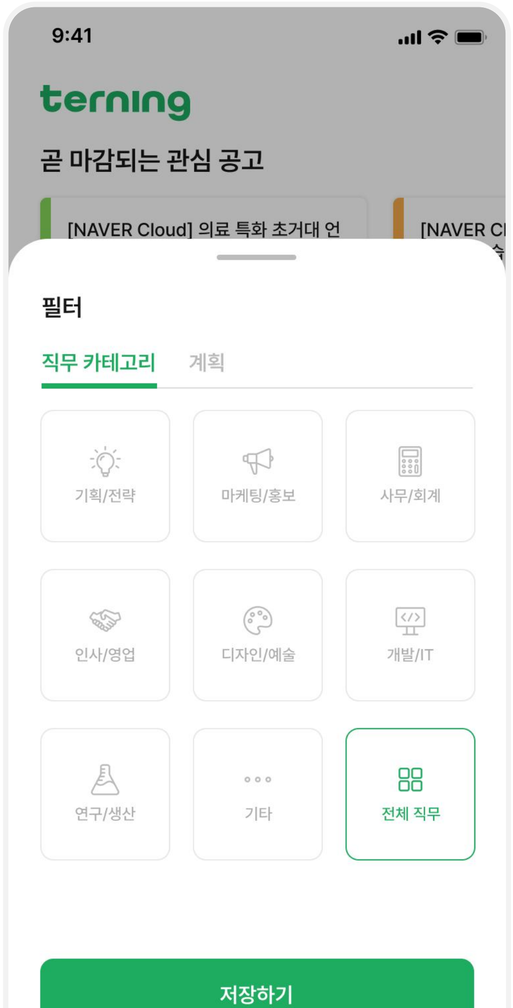
릴리즈 후 서비스 운영

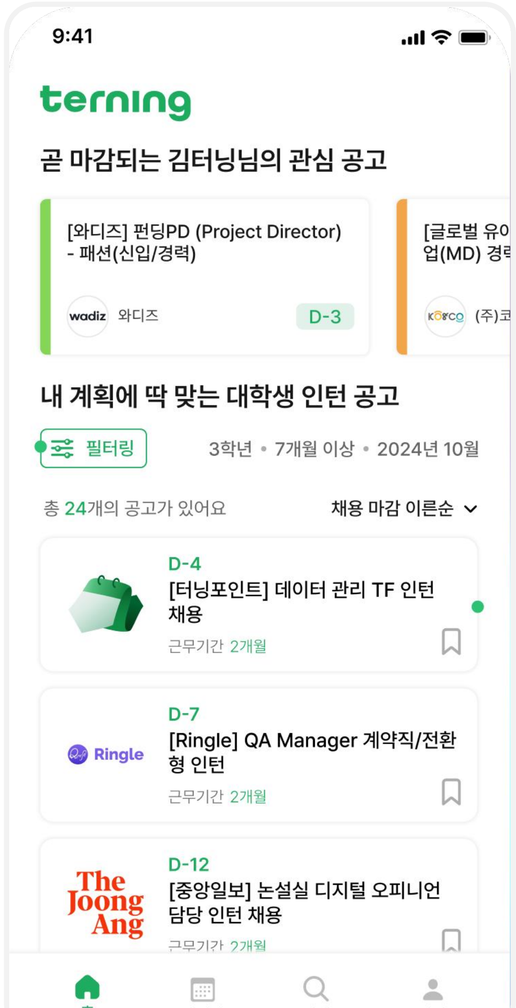
terning

유저테스트 UT를 4번 거듭하여,
1년 5개월 동안 사용자에게 꼭 필요한 기능을 추가했습니다

원하는 직무 공고만 보고 싶어요
필터링 없이 모든 공고를 보고 싶어요

직무 필터링 추가
: 직무 필터링 및 필터링 초기화 버튼 추가





terning

유저테스트 UT를 4번 거듭하여,
1년 5개월 동안 사용자에게 꼭 필요한 기능을 추가했습니다

새로운 인턴 공고가 올라오는지 알고 싶어요
지금 현재 인기있는 공고가 뭔지 알고 싶어요

푸시 알림 기능 추가
: 푸시알림을 누르면 상황에 맞는 화면으로 이동

📡 **터닝이가 관심 공고 일정을 알려드릴게요!** 9:41 AM
스크랩한 인턴 공고의 마감 일정, 정정교님의 캘린더에서 한눈에 확인해볼까요?

유저가 스크랩한 공고 리마인드
→ 캘린더로 이동

🎯 **터닝이가 새로운 인턴 공고를 골라왔어요!** 9:41 AM
정정교님의 원하는 직무와 계획에 맞춰, 딱 맞는 인턴 공고를 골라볼 수 있어요!

새로운 인턴 공고 업데이트 안내
→ 홈 화면으로 이동

👀 **다들 이 인턴 공고를 보고 있어요!** 9:41 AM
정정교님을 위해 터닝이가 인기 많은 인턴 공고를 가져왔어요! 놓치기 전에 살펴보세요.

조회수/저장수 높은 공고 확인
→ 탐색 뷰로 이동

Tickle

AI 기반 매크로 방지 '클린한 티켓팅' 플랫폼, 티클 Tickle

P95 응답시간 49% 단축,
Redis 요청 85% 감소시킨
AI 매크로 방지 티켓팅 플랫폼을 개발하여
프로젝트 116팀 중 우수상을 수상했습니다

소속 : SSAFY (삼성청년 SW · AI 아카데미) 부트캠프 14기 자율프로젝트

수상 : 본선 발표회 '우수상' 수상

구성 : 팀 프로젝트 (BE 2명, FE 2명, AI 2명)

기여도 : 인프라 100%, 백엔드 개발 50% 이상

상세 역할 : 인프라 & 백엔드 개발

Github : https://github.com/Tickle-ticket/Tickle

진행 기간 | 2026.04.06 ~ 2026.05.27 (8주)

Introduction

프로젝트 소개

30개 행동 데이터로 매크로를 AI가 실시간 탐지, 차단하는 공정한 티켓팅 플랫폼

기획 배경

매크로로 예매된 표가 최대 25배 가격으로 암표 거래되는 등 **티켓팅 시장의 봇 피해**가 증가했습니다.

기존 플랫폼들이 **매크로에 취약**한 문제를 해결하기 위해

AI로 **매크로를 실시간 탐지하고 차단하는 공정한 티켓팅 서비스**를 직접 만들고자 기획했습니다.

주요 기능 *담당 기능

- AI 매크로 탐지 - 사용자 행동 패턴을 AI가 분석해 매크로를 실시간으로 감지하고 차단
- ***실시간 대기열** - 동시 접속자가 몰려도 순번을 안정적으로 관리하고 실시간으로 안내
- ***좌석 선점** - 여러 명이 동시에 눌러도 딱 한 명만 선점, 실시간 좌석 현황 확인
- 결제 - 카카오페이 간편결제, 무통장 입금
- ***관리자 대시보드** - 매크로 현황, 대기열, 블랙리스트 실시간 관리

주요 기여

- 인프라 100% 설계 및 구축 - **5대 서버 역할 분리, Redis Sentinel HA, 전체 모니터링 스택**
- Virtual Thread + Redis Pipeline 도입으로 대기열 응답 **49%** 단축, Redis 호출 **85%** 감소
- 분산락 제거, InnoDB row lock 전환으로 좌석 선점 정합성 확보 (100 스레드 → 성공 1, 에러 0)
- BE-AI 탐지 연동 설계 - 30개 행동 데이터 수집, AI 추론, 블랙리스트 등록으로 **6가지 매크로 유형** 탐지

기술 스택

Backend	Java 21, Spring Boot 3.4.5, Spring Security, JPA
Real-time	Apache Kafka, SSE (Server-Sent Events), WebSocket
Database	MySQL 8.0, Redis 7 (Sentinel HA)
Auth	Kakao OAuth2, JWT
External	카카오페이 API, CoolSMS API
Infrastructure	Docker, Docker Compose, Jenkins, AWS ELB, AWS S3
Monitoring	Prometheus, Grafana, Loki, Tempo

주요 기술

Virtual Thread : SSE 장기 연결로 OS 스레드 고갈 문제 발생

→ **Virtual Thread 도입**으로 블로킹 비용 제거, 대기열 응답 49% 단축

Redis Pipeline : 대기자 1명당 Redis 7회 개별 호출로 성능 병목

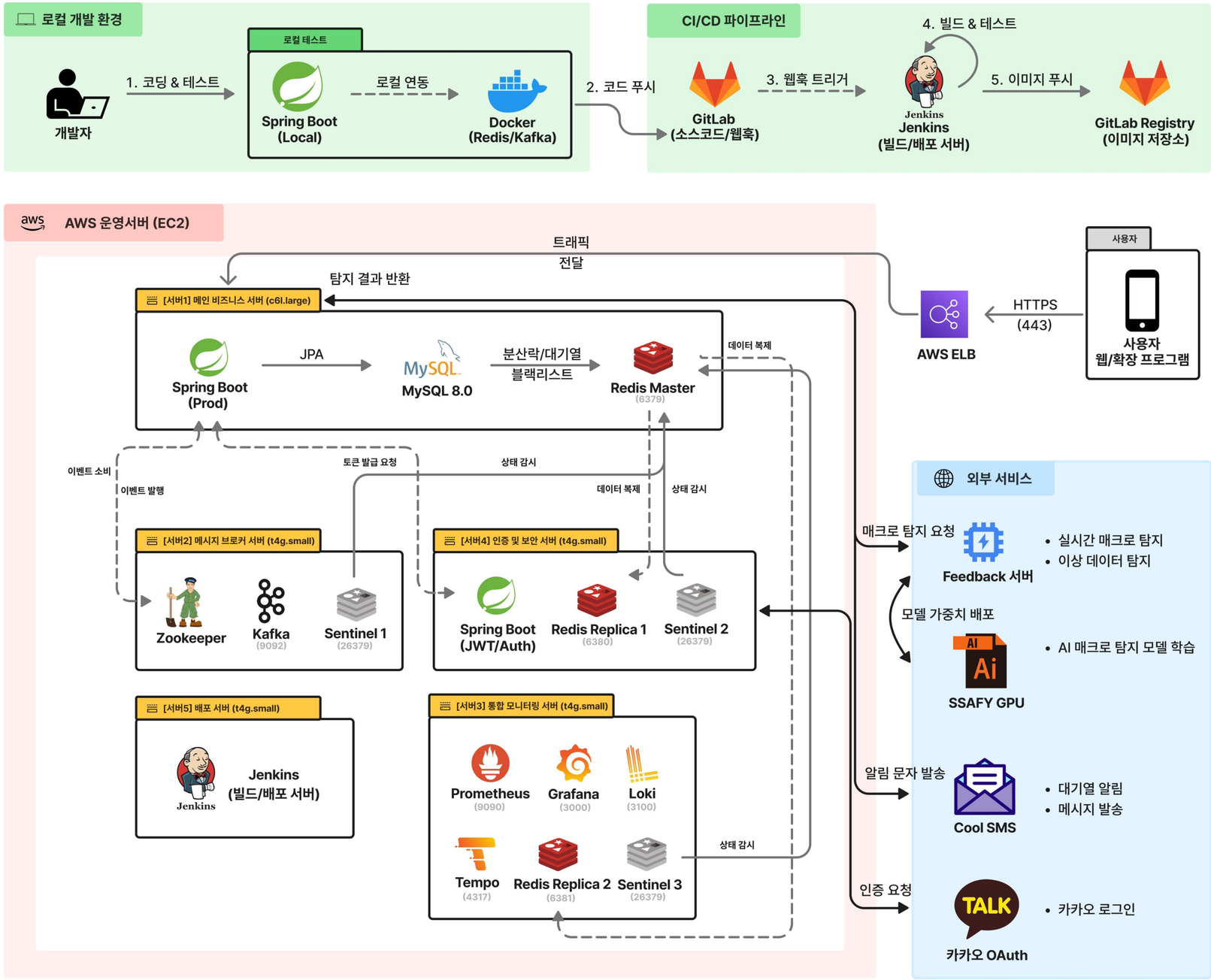
→ **단일 Pipeline 일괄 처리**로 호출 85% 감소 (7N회 → N+1회)

InnoDB row lock : 분산락을 회차 단위로 설정해 다른 좌석 요청까지 대기하는 문제

→ **좌석 단위 row lock**으로 대체, 병렬 처리 가능 및 정합성 유지

Redis Sentinel : 단일 노드 구성에서 Redis 장애 시 대기열, 선점 서비스 동시 중단 위험

→ **Master 1 + Replica 2 + Sentinel 3대 구성**으로 5초 내 자동 페일오버



인프라 아키텍처 설계

가용성과 장애전파를 고려하여 전체 서비스 인프라를 직접 설계했어요

대기열 + 실시간 동기화 (Kafka + SSE + Redis)

- Kafka로 대기열 진입 요청을 비동기 적재하고 Redis Sorted Set으로 순번을 관리해 SSE로 클라이언트에 실시간 Push하는 구조를 구현했습니다. 좌석 상태 변경은 DB 커밋 이후 비동기 브로드캐스트를 적용해 선점 API의 응답 지연을 차단했습니다.

인증 서비스 분리 + Redis Sentinel HA

- 인증 서버를 server4에 분리해 JWT 발급/Rotation/블랙리스트를 담당했습니다. 메인 서버는 로컬 서명 검증 + Redis 블랙리스트 1회 조회 방식으로 매 요청 RTT를 제거했습니다. Redis Sentinel 3대 구성으로 페일오버를 보장하도록 구성했습니다.

모니터링 + CI/CD 자동화 (Prometheus + Grafana + Jenkins)

- server3에 Prometheus + Grafana + Loki + Tempo 풀스택을 구축했습니다. 분산 트레이싱과 Grafana Alerting으로 Mattermost 알림을 연동하고, Jenkins는 모노레포 경로 감지로 BE/Auth 빌드를 병렬화해 배포시간을 단축했습니다.

문제 해결 01 : Virtual Thread · Redis Pipeline 도입을 통한 대기열 응답 속도 개선

Tickle

200명이 동시에 대기열에 진입해도 응답시간 49% 단축, Redis 호출 85% 감소를 달성했어요

*889ms → 454ms, 7N회 → N+1회로 개선

Steps 단계별 개선 과정

1단계 : SSE 장기 연결로 OS 스레드 고갈

(고민) WebFlux 전환 vs Virtual Thread vs SSE timeout 단축

(결정) OS 스레드에 종속되지 않는 Virtual Thread 도입 결정.

2단계 : 가상 스레드 급증으로 1GB 힙 OOM 발생

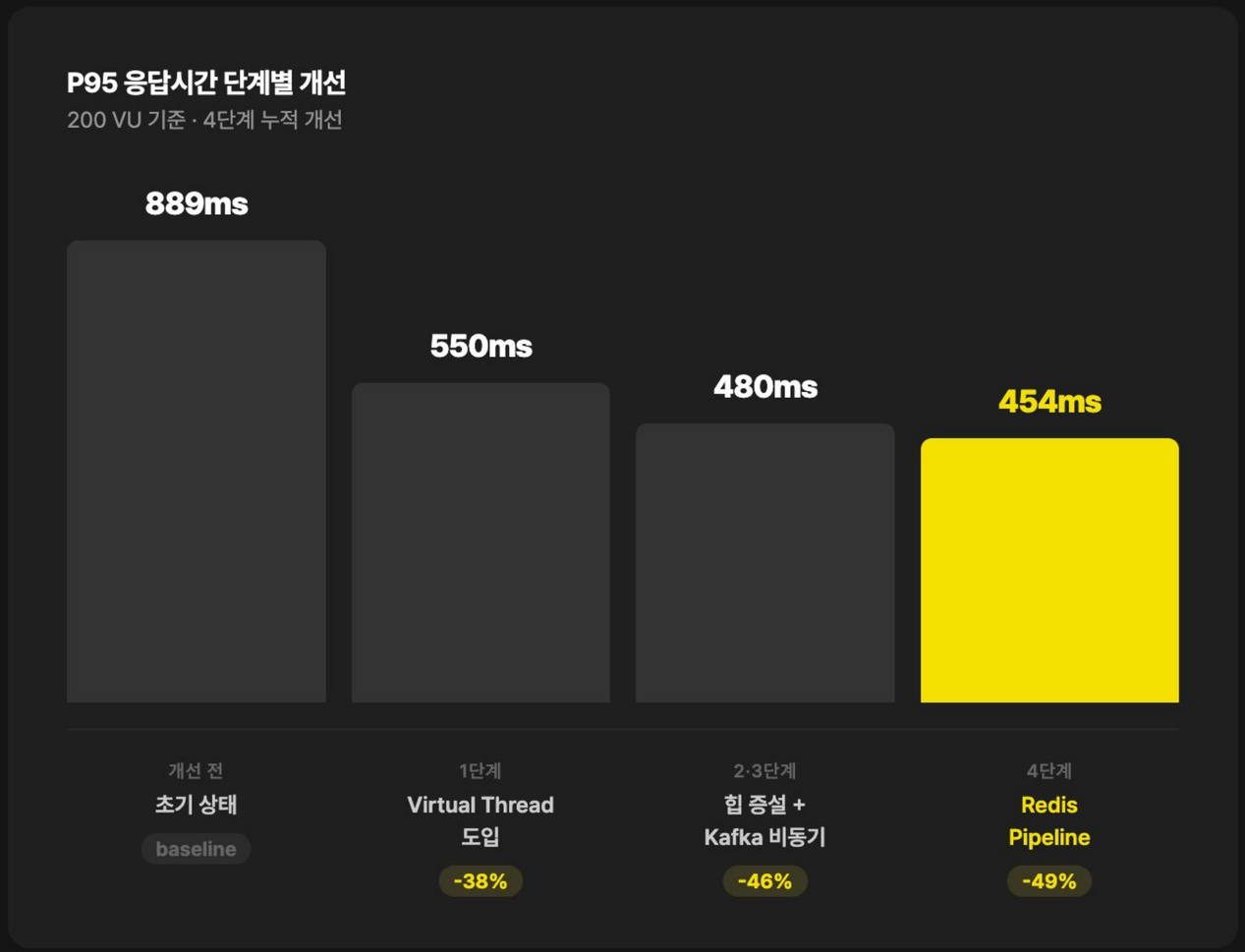
(결정) 힙 1GB → 1.75GB 증설 / SSE timeout 30분 → 3분 단축.

3단계 : Kafka 동기화 블록이 가상 스레드 점유

(결정) Kafka 동기 ack → 비동기 콜백 전환으로 스레드 pinning 현상 제거.

4단계 : 스케줄러가 대기자 1명당 Redis 7회 개별 호출

(결정) 쓰기 명령 7N회를 단일 Pipeline으로 일괄 처리



문제 해결 02 : Redis Sentinel HA 구성으로 대기열·선점 서비스 단일 장애점 제거

Tickle

Redis Sentinel을 도입해 서비스 중단 없는 고가용성 구조를 설계했어요

*Master 1 + Replica 2 + Sentinel 3대, Quorum=2 구성

Problem 문제

Redis 단일 노드 구성에서 장애 발생 시
대기열, 좌석 선점, 토큰 검증이 동시 중단.

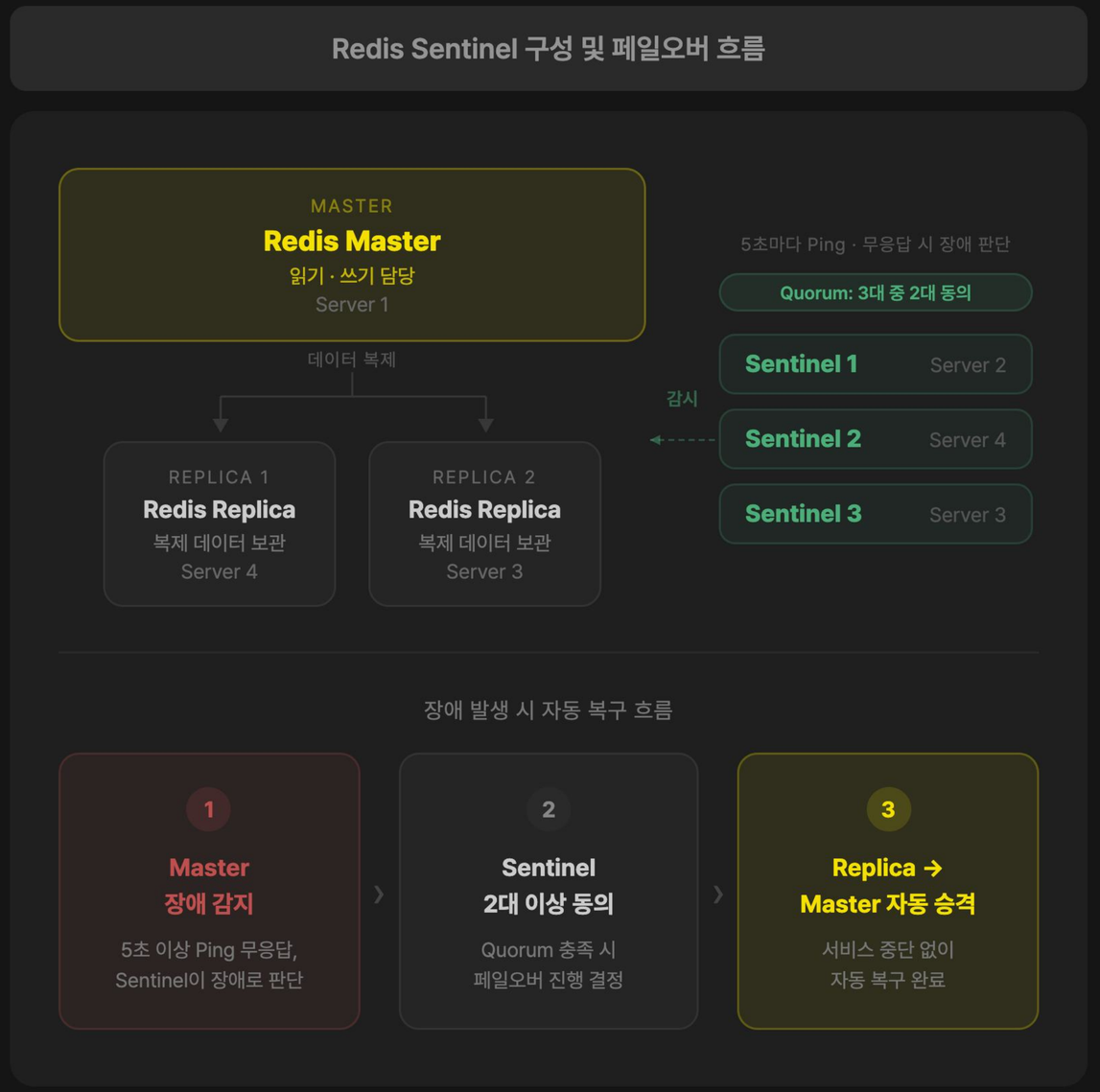
Reasons 의사결정 과정

(고민) Redis Cluster vs Sentinel

(결정) 데이터 샤딩이 불필요한 규모에서 운영 복잡도가 낮은 Sentinel로 결정

Solution 해결

Sentinel 3대로 5초 내 Master 장애를 감지하고
Quorum 2대 이상 동의 시 Replica를 Master로 자동 승격.



문제 해결 03 : 티켓팅 전 구간 행동 데이터 수집 → AI 탐지 → 실시간 차단 연동

Tickle

30개 행동데이터로 학습한 AI가 매크로를 판별하고, BE가 즉시 블랙리스트로 차단하는 파이프라인을 구축했어요

1 티켓팅 플로우

① 예매 클릭 → ② CAPTCHA → ③ 좌석 선택 → ④ 결제 → ⑤ 예매 확정

2 행동 데이터 수집 30개 피처

클릭 시간 · 마우스 이동 · 요청 간격 · 이동 횟수 등

BE 백엔드 역할

- 행동 데이터 수집·적재
- AI 서버 추론 요청
- Redis 블랙리스트 관리
- 관리자 대시보드 API

AI AI 역할

- 30개 피처 기반 추론
- 이진 분류 모델
- 임계값 설계

3 AI 추론 & 판정

AI SERVER

모델 추론

- 30개 피처 벡터 분석
- 이진 분류 모델 판정
- 실시간 추론 응답

정상

사용자 통과

- 예매 플로우 계속 진행
- 접근 허용

매크로

즉시 차단

- Redis 블랙리스트 등록
- 이후 요청 전면 차단
- 관리자 대시보드 기록

문제 해결 04 : 코드리뷰, 문서화, 발표까지 - 자발적으로 팀 전반에 기여

코드 품질, 지식 공유, 일정 관리, 발표까지
팀에 필요한 것을 직접 채우며 팀 전반에 기여했어요